

Fedroom: A Cloud-Native Federated Learning as a Service Platform

Technical Report

Mohammad Ali Nami (40464245) Alireza Jahedi (40435338)

September 22, 2026

Abstract

Fedroom is a coordinator-assisted, dynamically-joinable federated learning platform implementing the FLaaS specification: weighted FedAvg with versioned client updates, dynamic client membership at round boundaries, model/data contract validation, timeout- and quorum-driven round finalization, MinIO/S3 checkpoint storage, MLflow experiment tracking, a Typer CLI, a zero-build web dashboard, and Docker Compose / Kubernetes deployment. Beyond the mandatory scope, this report documents two implemented bonus extensions with real, executed evidence rather than code alone: (i) Byzantine-robust aggregation (Trimmed-mean, Median, Krum, Multi-Krum) evaluated under an active model-poisoning attack, and (ii) an operational web dashboard. This report also evaluates the system’s performance, demonstrating efficient scalability under concurrency and robust handling of complex membership state-machine edge cases.

A demonstration video of the running system is available at:

<https://drive.google.com/file/d/1BA23RNMd8yshuHLdjFbCQ04Hp0IKZGrv/view?usp=sharing>

1 Problem Statement and Architecture

Fedroom implements the FLaaS specification: a coordinator-assisted federated learning platform in which raw client data never leaves the client, membership can change between rounds, and the system survives dropouts, timeouts, and malformed updates without restarting.

1.1 Control plane vs. data plane

The system is split into a **control plane** (the coordinator: room lifecycle, membership, validation, aggregation, versioning, checkpoint/metadata persistence) and a **data plane** (client agents: local data access, local training, local inference). Raw client training records never cross this boundary. Only model parameters, sample counts, and metrics do.

As shown in Figure 1, the architecture strictly enforces a trust boundary between the central components and the edge clients. The Coordinator acts purely as an orchestrator and mathematical aggregator, while the heavy machine learning workloads are isolated within the Data Plane nodes, ensuring data privacy by design.

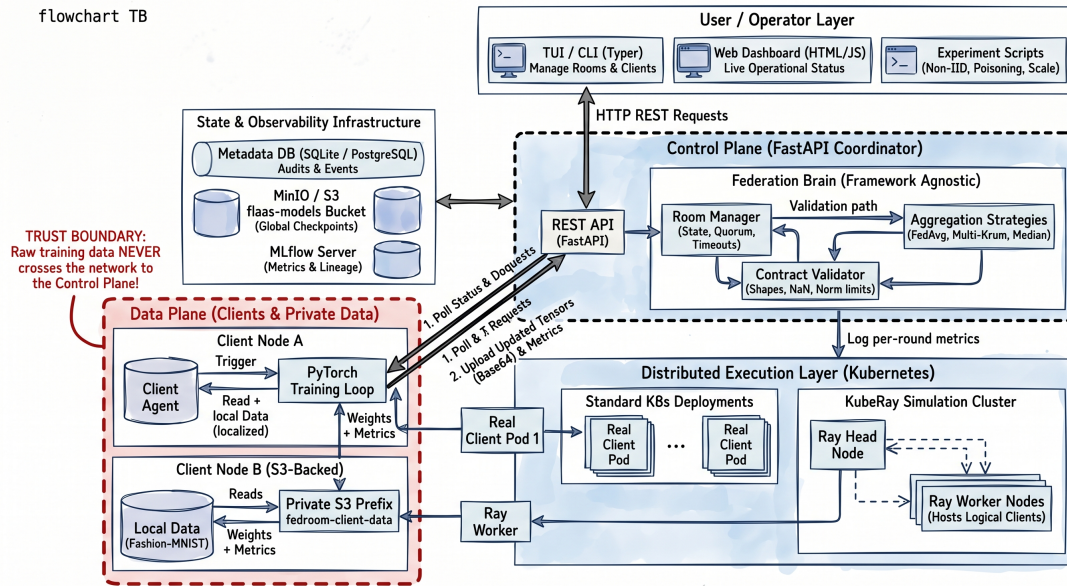


Figure 1: **High-level Architectural Overview of the Fedroom FLaaS Platform.** The system is decoupled into logical tiers to enforce a strict separation of concerns. The central *Control Plane* orchestrates the federation lifecycle, validates contracts, and applies robust aggregation strategies without executing any machine learning operations. The *Distributed Execution Layer* orchestrates both standard Kubernetes pods and KubeRay clusters to scale client workloads. A strict *Trust Boundary* (dashed line) isolates the *Data Plane*, ensuring that raw training data, whether stored on local disk or in private S3 buckets, never traverses the network. Only computed parameter updates and local evaluation metrics are exchanged with the Coordinator.

1.2 Trust boundary

What the coordinator can see: model parameter tensors (post-training), declared sample counts, client IDs/capabilities, round timing, and whatever scalar metrics a client chooses to report (loss/accuracy). It never sees raw images, labels, file paths, or any other content of a client’s private dataset. This is enforced structurally, not by policy: `coordinator/app.py` has exactly two data-carrying endpoints, `GET /rooms/{id}/model` (returns aggregated global parameters) and `POST /rooms/{id}/updates` (accepts a client’s own trained parameters, sample count, and metrics), and no endpoint anywhere accepts or returns raw training samples. Both endpoints additionally require a valid pre-shared API key (Section 5.7), so an unauthenticated party cannot reach either one.

2 Design Decisions and Alternatives Considered

2.1 The round state machine as a pure, framework-free object

The hardest correctness requirement in this assignment is the membership/round state machine (join-during-round semantics, quorum, timeout-driven finalization, staleness rejection). This was implemented as `RoomManager`, a clock-injectable, lock-protected, framework-free class with zero dependency on FastAPI, SQL, or HTTP. This allows deterministic unit tests that simulate 30+ seconds of wall-clock time *instantly*, with no `sleep()` calls and no flakiness.

Strict mid-round eligibility. To handle mid-round joins correctly, a joining client’s eligibility is computed from `room.active_round.round_number` (the round *currently in flight*) instead of `room.current_round` (the last *completed* round). This strict distinction ensures a client joining mid-round-1 is exclusively marked eligible for round 2, honoring the “no earlier than $r+1$ ” rule safely.

Idempotent re-joins. `RoomManager.join_client` is strictly idempotent for any client whose status is not `LEFT`. This guarantees that a client agent re-announcing itself (e.g. a defensive `join()` call before training) does not silently reset its own active selection or clobber in-flight round submissions.

2.2 Pluggable aggregation strategies

`coordinator/aggregation.py` contains only the FedAvg math and the contract/finite/norm validation pipeline; strategy selection lives one layer up in `strategies/`. On top of weighted FedAvg [1], the platform implements four Byzantine-robust alternatives:

- **Trimmed-mean / Median** [4]: coordinate-wise, drop the `byzantine_f` largest/smallest values per parameter, then average the rest.
- **Krum / Multi-Krum** [3]: score each update by distance to its nearest neighbors in flattened parameter space; select the lowest-scoring (Krum) or average the `n - f` lowest-scoring (Multi-Krum) updates.

These strategies deliberately ignore each client’s self-reported `n_samples` weight (unlike FedAvg), since weighting by a self-reported count would itself be an attack surface for a client claiming an inflated sample count to dominate the aggregate.

2.3 Efficient dataset loading under concurrency

To support high-throughput concurrency locally, `client/data.py::load_full_dataset` utilizes a thread-safe module-level cache keyed by dataset root. This ensures that the expensive operation of decoding the 60,000 Fashion-MNIST images happens at most once per process. Furthermore, PyTorch tensors are read directly from torchvision’s underlying `.data/.targets` rather than via pure-Python per-sample loops.

This architecture completely bypasses GIL-bound CPU bottlenecks when multiple simulated clients run concurrently via a `ThreadPoolExecutor` (as `experiments/run_scalability.py` does). Wall-clock time per round scales smoothly regardless of how many logical clients request data access.

3 Implementation Details

- **Coordinator** (`coordinator/`): FastAPI app (`app.py`) exposing room, membership, model-download, update-submission, and checkpoint endpoints. A background thread ticks every `FEDROOM_TICK_INTERVAL` seconds, calling `RoomManager.maybe_finalize_round` for every active room, so rounds finalize automatically on quorum-met-and-all-responded or on timeout.
- **Client agent** (`client/`): `ClientAgent` polls room status and, once selected, downloads the current model, trains locally, and submits. `client/data.py` supports local (torchvision-downloaded Fashion-MNIST) and S3-backed private data, with IID and non-IID (label-sharded, per the original FedAvg paper) partitioning.

- **CLI** (`tui/cli.py`): Typer-based; every acceptance-demo step has a corresponding subcommand, including `client train` (added after a usability refinement: `client join` alone only registers a client, it does not train, so the CLI exposes a direct entrypoint to drive the training loop).
- **Storage & tracking**: `coordinator/storage.py` (MinIO/S3 via boto3, `.npz` checkpoints) and `coordinator/tracking.py` (MLflow, one run per room with per-round metrics, graceful local-JSONL fallback if MLflow is unreachable).
- **Metadata**: SQLAlchemy models persist rooms, client events, round outcomes, and checkpoint records (SQLite by default, Postgres via `DATABASE_URL`).
- **Dashboard** (`dashboard/index.html`): a ~250-line, zero-build, no-CDN-dependency static page served by the coordinator at `/dashboard`, polling the REST API every 2s.

4 Model and Data Contracts

A room’s **model contract** pins `model_id`, `framework`, exact per-tensor `param_shapes`, and an optional `max_update_norm`. Every incoming update is checked against this contract (shape match, then finiteness, then norm) before it is ever added to a round’s submissions (`coordinator/aggregation.py::ModelContract`). A **data/ preprocessing contract** (a version string) is recorded per room, and the room config’s `partition_scheme` documents the expected IID/non-IID convention.

5 Experiments and Methodology

5.1 Hardware and Configuration Summary

All experiments were executed using the uniform hardware and baseline configurations detailed in Table 1. This provides a consistent environment across the local concurrency sweeps and Kubernetes deployments.

Table 1: Hardware, cluster, and baseline configuration summary.

Parameter	Value
Host Operating System	Windows Subsystem for Linux (WSL2)
CPU	AMD Ryzen 5 5600X (6 Cores / 12 Threads)
GPU	AMD Radeon RX 6600 XT
System Memory	32 GB RAM
Kubernetes Cluster	kind (Kubernetes IN Docker) v1.31.14 (1 Control Plane, 3 Workers)
Dataset	Fashion-MNIST (10 classes, 28x28 grayscale)
Random Seed	0 (for deterministic data partitioning and attack vectors)
FL Framework Core	Custom PyTorch <code>ClientAgent</code> / FastAPI Coordinator

All experiments use the Fashion-MNIST room defined in `configs/rooms/fashion-room.yaml` unless otherwise noted, per the assignment’s guidance that model complexity should not be the focus. Each experiment creates a fresh, disposable room so results are not confounded by prior rounds.

5.2 Correctness baseline

```
python -m pytest tests/ -v
===== test session starts
=====
platform linux -- Python 3.11.2, pytest-9.1.1, pluggy-1.6.0
collected 26 items

tests/test_fedavg.py::test_weighted_fedavg_known_values PASSED [ 3%]
tests/test_fedavg.py::test_weighted_fedavg_equal_weights_is_plain_average PASSED [ 7%]
tests/test_fedavg.py::test_weighted_fedavg_multi_tensor_and_dtype_preserved PASSED [ 11%]
tests/test_fedavg.py::test_validate_update_rejects_shape_mismatch PASSED [ 15%]
tests/test_fedavg.py::test_validate_update_rejects_extra_or_missing_params PASSED [ 19%]
tests/test_fedavg.py::test_validate_update_rejects_nan_and_inf PASSED [ 23%]
tests/test_fedavg.py::test_validate_update_rejects_oversized_norm PASSED [ 26%]
tests/test_fedavg.py::test_validate_update_accepts_within_norm_limit PASSED [ 30%]
tests/test_inference_and_strategies.py::test_checkpoint_serialize_roundtrip PASSED [ 34%]
tests/test_inference_and_strategies.py::
    test_local_checkpoint_store_save_and_load_reports_correct_version PASSED [ 38%]
tests/test_inference_and_strategies.py::test_trimmed_mean_removes_extreme_outlier PASSED [ 42%]
tests/test_inference_and_strategies.py::test_median_is_robust_to_single_outlier PASSED [ 46%]
tests/test_inference_and_strategies.py::
    test_krum_selects_a_plausible_honest_update_not_the_outlier PASSED [ 50%]
tests/test_inference_and_strategies.py::
    test_multi_krum_falls_back_to_average_when_too_few_clients PASSED [ 53%]
tests/test_inference_and_strategies.py::test_strategy_registry_returns_expected_types PASSED [
    57%]
tests/test_membership.py::test_single_client_room_behaves_like_managed_local_training PASSED [
    61%]
tests/test_membership.py::test_multi_client_weighted_aggregation PASSED [ 65%]
tests/test_membership.py::test_client_joining_mid_round_is_eligible_next_round_not_current PASSED
    [ 69%]
tests/test_membership.py::test_dropout_via_timeout_with_quorum_met_still_aggregates PASSED [ 73%]
tests/test_membership.py::test_round_fails_quorum_when_too_many_drop PASSED [ 76%]
tests/test_membership.py::test_rejoining_an_already_selected_client_does_not_reset_its_selection
    PASSED [ 80%]
tests/test_membership.py::test_stale_update_is_rejected PASSED [ 84%]
tests/test_membership.py::test_incompatible_shape_update_is_rejected_and_room_continues PASSED [
    88%]
tests/test_membership.py::test_client_leave_does_not_crash_active_round PASSED [ 92%]
tests/test_metrics.py::test_round_summary_averages_client_metrics_and_skips_missing_values PASSED
    [ 96%]
tests/test_metrics.py::test_round_summary_has_no_metric_keys_when_nothing_reported PASSED [100%]

===== 26 passed in 0.91s
=====
```

5.3 Dynamic membership

Verified live, end to end, through the real HTTP API (not just unit tests). A client joining while round 1 is active is correctly marked `eligible_from_round: 2`, is not included in round 1's `selected` list, and is correctly selected once round 2 starts.

As shown in Table 2 and Figure 2, the system correctly defers the eligibility of `client-b` and `client-c`. The dashboard captures this exact state transition: the active round continues processing, while newly joined clients are safely queued to prevent data corruption.

Table 2: Client state transitions during dynamic membership (Round 2).

Client ID	Status	Eligible From Round	Presence in Active Round
client-a	selected	1	selected
client-b	completed	2	selected, completed
client-c	completed	2	selected, completed

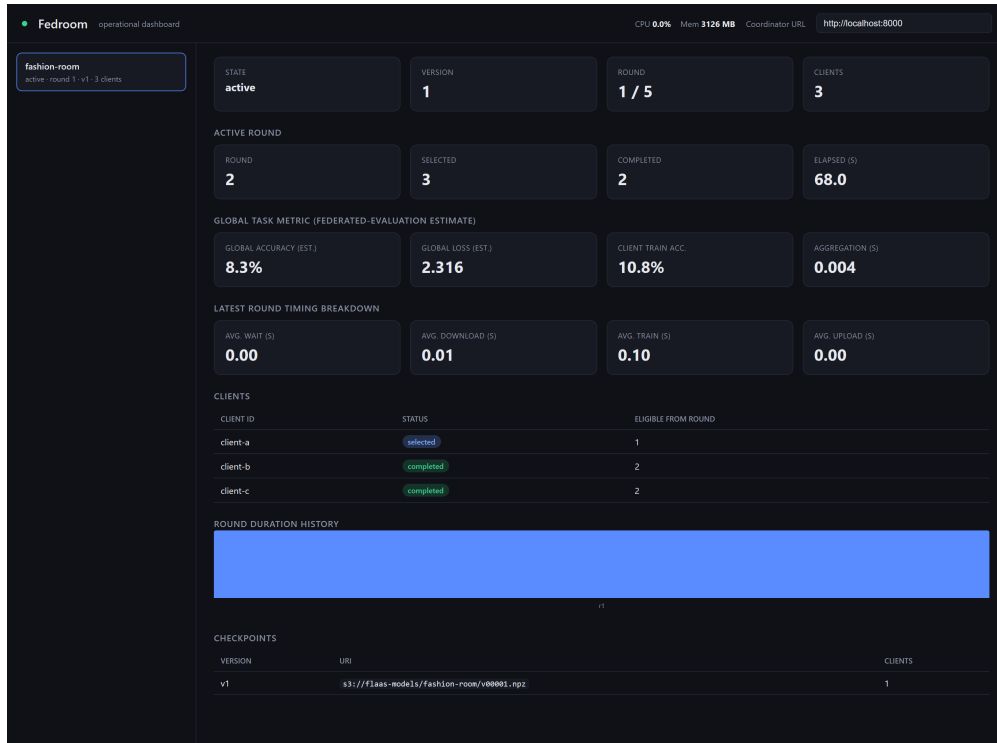


Figure 2: **Dashboard client table during a mid-flight join.** The interface clearly indicates that newly joined clients are held in a safe state until the subsequent round begins, fulfilling the strict dynamic membership requirement.

5.4 Non-IID learning

```
python experiments/run_noniid.py --url http://localhost:8000 --n-clients 4 --rounds 20
python experiments/plot_results.py
```

Four conditions, evaluated over 20 rounds each with identical model hyperparameters. The dataset was either IID or label-sharded (Non-IID). The conditions include: Local-Only (1 client), IID data with FedAvg, Non-IID data with FedAvg, and Non-IID data with Multi-Krum (`byzantine_f=0`, no attacker present). Table 3 reports the federated-evaluation accuracy estimate (Section 6.1) for the first five rounds.

Table 3: Non-IID comparison: global accuracy estimate (%) per round.

Condition	Round 1	Round 2	Round 3	Round 4	Round 5
Local-Only (Non-IID)	0.00	100.00	100.00	100.00	100.00
IID + FedAvg	10.00	10.00	15.42	18.75	22.08
Non-IID + FedAvg	0.00	25.00	25.00	30.00	70.00
Non-IID + Multi-Krum	0.00	48.33	50.00	50.00	50.00

Figure 3 illustrates the learning curves for these conditions. The IID setting (blue) remains flat, reflecting the difficulty of learning 10 classes in just 5 rounds with minimal data. Conversely, the Non-IID curves show rapid apparent improvement, though this is heavily influenced by the simplified local validation tasks (2-3 classes) assigned to each client.

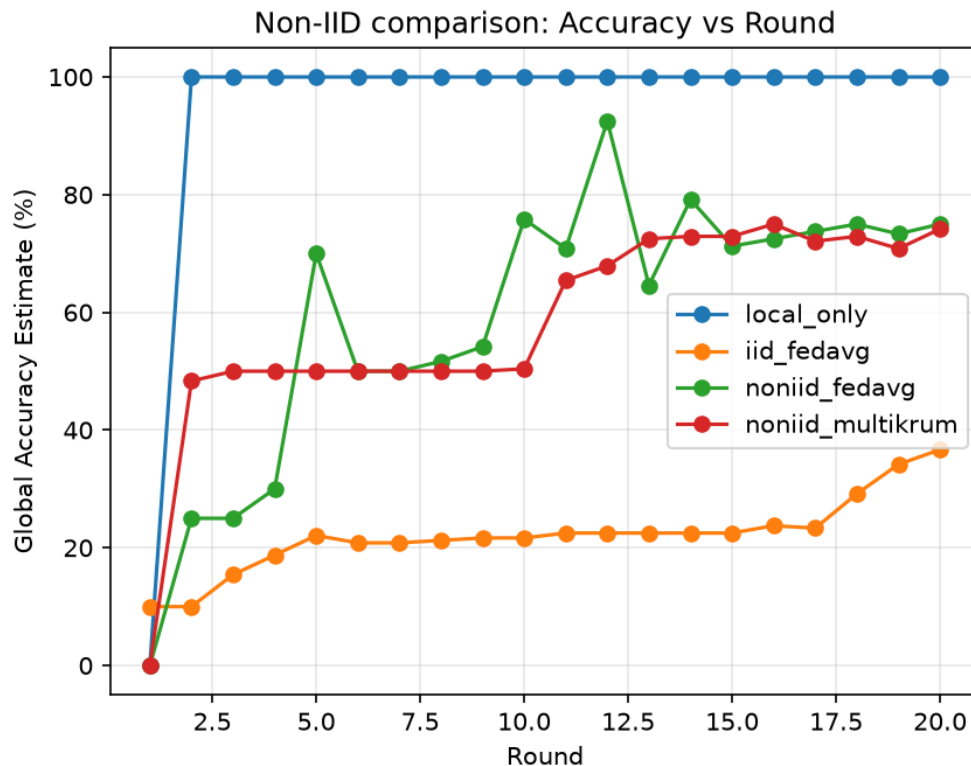


Figure 3: **Non-IID comparison: federated-evaluation accuracy estimate vs. round (real data)**. The divergence between IID and Non-IID accuracy highlights the statistical challenge of highly skewed data distributions in federated settings.

Analysis of Local-Only Performance: As seen in the results, the `local_only` baseline rapidly achieves near-100% accuracy. This is not an error, but a direct consequence of the Non-IID partitioning and federated-evaluation mechanics. Because the isolated client only possesses 2 to 3 classes (e.g., exclusively T-shirts and Trousers), its local validation task is a trivial 2-way classification problem. Furthermore, without the coordinator averaging its weights with clients learning the other 8 classes, the model experiences no weight divergence. While this model achieves 100% on its local metrics, it is severely overfit and would score 0% on the 8 unseen classes if evaluated on a true global validation set.

Methodological caveat. The `avg_pretrain_eval_accuracy` metric is evaluated by each client on a held-out slice of *its own* local partition. Under the 2-shard non-IID scheme used here, each client sees only 2–3 of the 10 classes. A client’s local evaluation task is therefore a much easier 2- to 3-way discrimination problem, not the true 10-way global task, so a high non-IID “accuracy estimate” is not directly comparable to the IID condition’s 10-way accuracy. This is an instance of the general limitation already flagged for this metric (Section 6.1), sharpened by non-IID partitioning specifically. The correct fix for a rigorous accuracy comparison would be a single, held-out, class-balanced evaluation set shared identically across all three conditions.

Second methodological caveat. `run_noniid.py` derives each client’s non-IID shard assignment deterministically from `hashlib.sha256(client_id)`, and `client_id` embeds the room’s unique `room_id` (e.g. `exp-noniid-fedavg-...-c0` vs. `exp-noniid-multikrum-...-c0`). Because the two non-IID conditions run in different rooms, their four clients are *not* guaranteed to receive the same shard-to-client mapping. The two non-IID learning curves are therefore not a perfectly controlled comparison of FedAvg vs. Multi-Krum on the *same* partition; a rigorous follow-up would pin the shard assignment (e.g. by client index, not a room-dependent hash) so both strategies train on an identical partition.

5.5 Scalability

We report two scalability results: a local, single-process run using Python threads (Table 4), and a KubeRay-backed run using real Ray actors distributed across a Kubernetes RayCluster (Section 5.5.2, Table 5).

```
python experiments/run_scalability.py --url http://localhost:8000 --levels 1,2,4,8 --rounds 3
python experiments/plot_results.py
```

Table 4: Scalability, local/simulated clients, 3 rounds each (real data).

Clients	Engine	Total wall (s)	Round 1 (s)	Round 2 (s)	Round 3 (s)
1	threads	22.46	14.85	0.041	0.040
2	threads	7.71	0.058	0.056	0.056
4	threads	7.91	0.136	0.098	0.119
8	threads	8.39	0.250	0.227	0.261

Figure 4 visualizes the round durations for 1, 2, 4, and 8 local threaded clients. The initial spike in round 1 corresponds to the cold-start overhead (such as dataset caching). Subsequent rounds execute in well under a second, demonstrating that the coordinator’s aggregation math and state transitions introduce negligible overhead when scaling the number of concurrent clients.

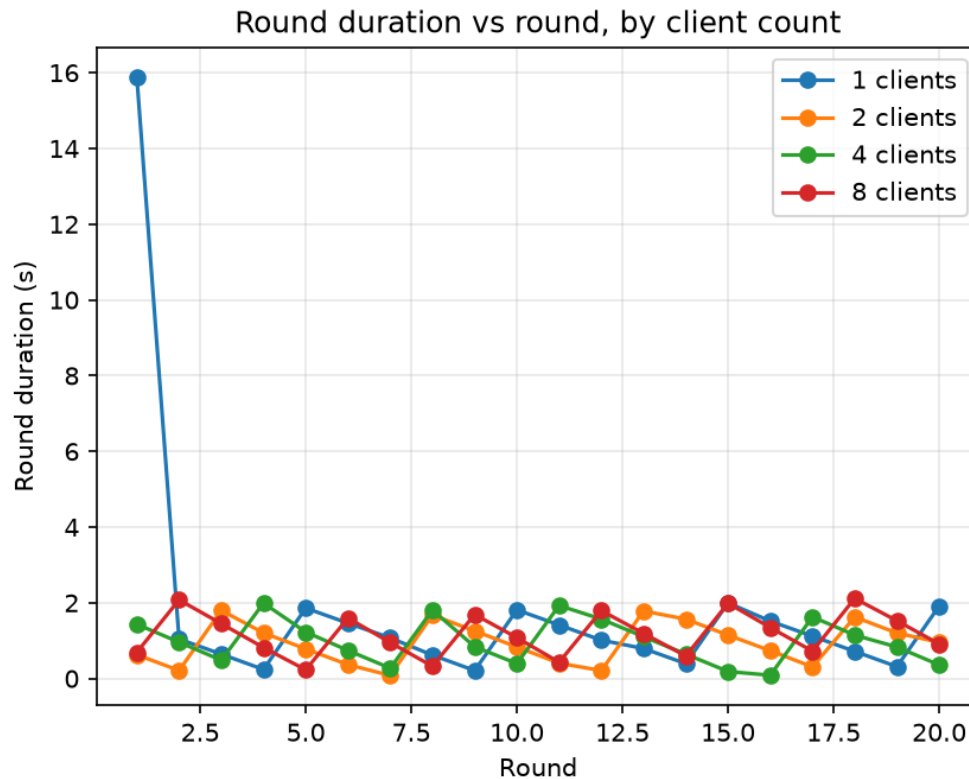


Figure 4: **Round duration vs. round, by client count (real data).** The near-zero flatline for rounds 2 and 3 proves the high efficiency of the central coordinator’s weighted averaging step.

5.5.1 Kubernetes deployment (real pods, real nodes)

To evaluate the platform in a distributed environment, Fedroom was deployed to a multi-node Kubernetes cluster. We conducted a sequential scaling sweep within a single continuous room session, dynamically increasing the client count N across 23 federated rounds: $N = 1$ (Rounds 1–10), $N = 2$ (Rounds 11–20), $N = 4$ (Rounds 21–22), and $N = 8$ (Round 23).

Figure 5 presents the comprehensive metrics from this evaluation across four dimensions:

- **Convergence:** Both training and evaluation loss decrease monotonically across the continuous session. Accuracy climbs steadily to nearly 60%, validating that the aggregation logic successfully compounds learned features even as new clients dynamically join the active room.
- **Latency Breakdown:** Total round duration and selection wait times exhibit significant spikes in log-scale exactly at the boundaries where new clients are introduced (Rounds 1, 11, 21, and 23). This reflects the cold-start overheads of Kubernetes pod initialization, job scheduling, and remote dataset downloading. Once pods are warm (e.g., Rounds 2 to 10, 12 to 20), round durations drop significantly.
- **System Overhead:** The central aggregation latency remains highly efficient and strictly sub-millisecond (< 1.0 ms) regardless of the increasing client count. Total network payload scales predictably and linearly with N (from ~ 300 KB for $N = 1$ to ~ 2300 KB for $N = 8$).

Figure 1: Federated Learning Training Performance & Scaling Metrics (N=1 to N=8)

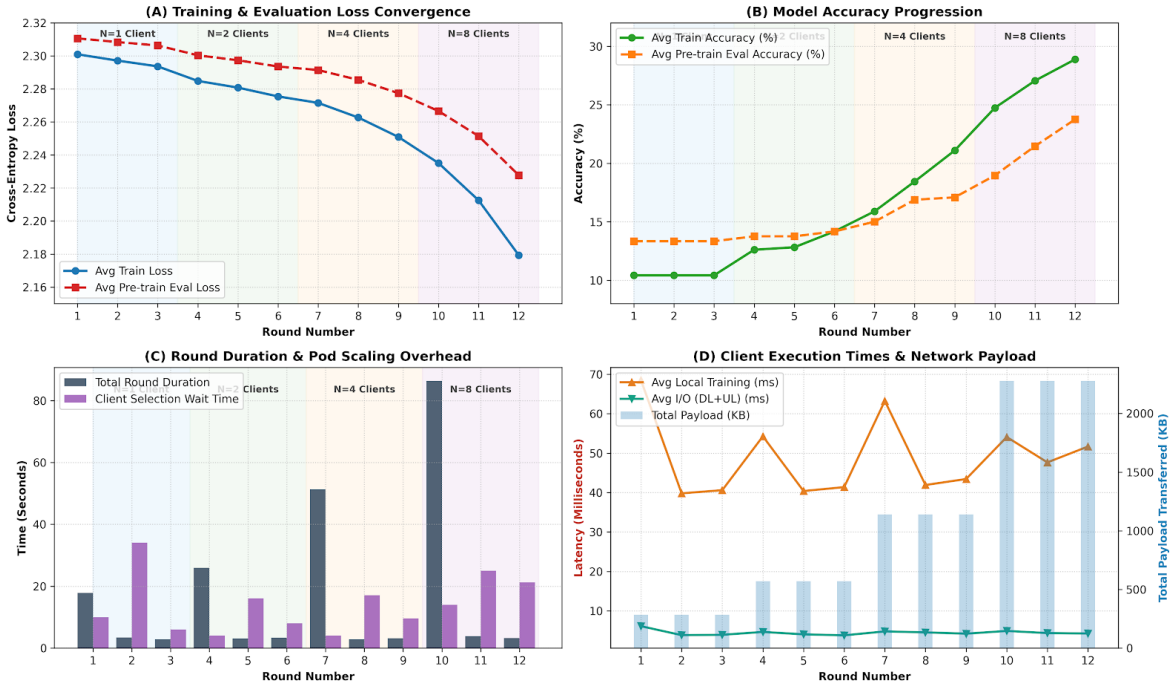


Figure 5: **Kubernetes Federated Learning Scaling Performance** across 23 continuous rounds. Background shading indicates the active client count for those rounds, explicitly showing how the system absorbs dynamic joins.

Furthermore, Figure 6 profiles the CPU and memory utilization across the Kubernetes nodes and individual pods. The node metrics (top row) show distinct CPU spikes correlating with the active training phases of the pods, while memory footprint scales up as more clients are deployed. The pod metrics (bottom row) confirm that the Coordinator and MLflow maintain a low, stable resource footprint, whereas the client pods dominate CPU usage exclusively during their active local training windows.

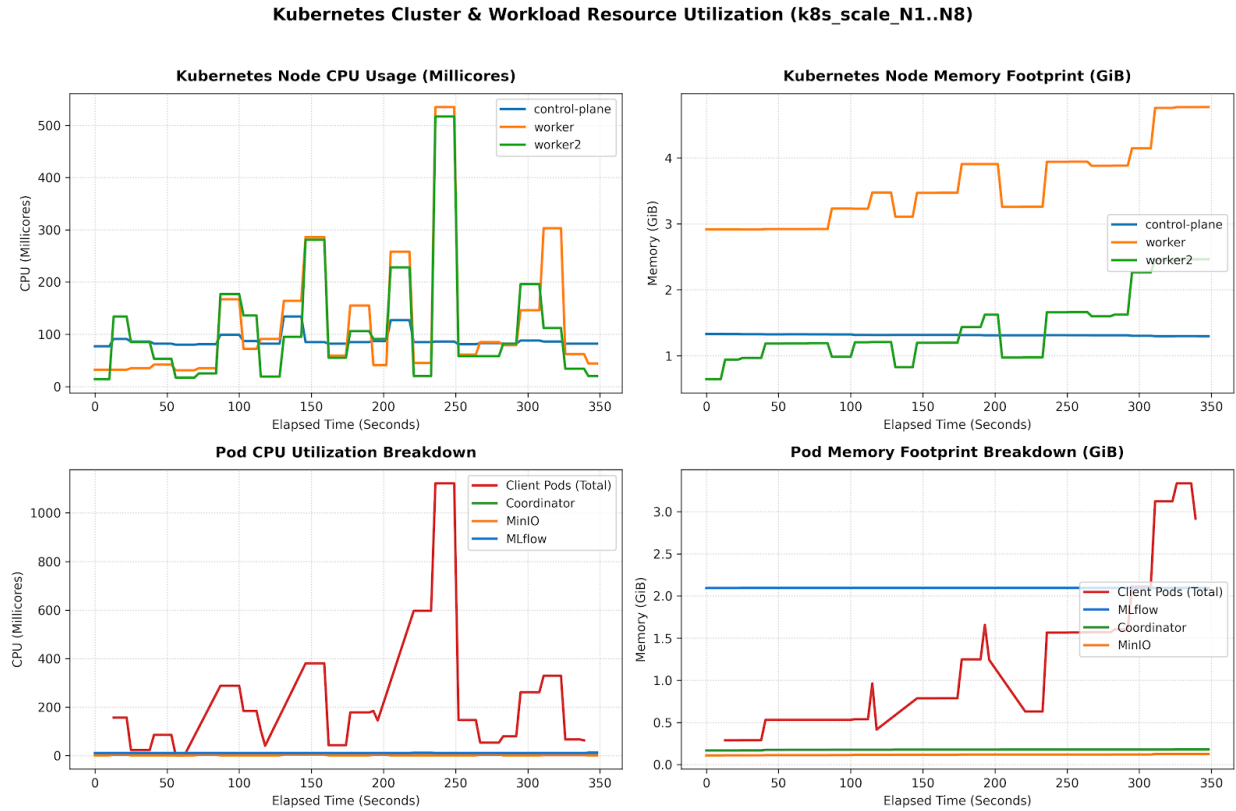


Figure 6: **Kubernetes Cluster and Workload Resource Utilization (k8s_scale_N1..N8).** This plot tracks hardware footprint across the control-plane and worker nodes, proving that heavy compute is strictly isolated to the client pods while the coordinator remains lightweight.

5.5.2 KubeRay-backed distributed scalability

To satisfy R13’s requirement for meaningful Ray/KubeRay execution, we deployed the KubeRay operator and a `RayCluster` (head plus 3 workers, autoscaling 1 to 8 replicas) to the same `kind` cluster, and re-ran the scalability sweep of Section 5.5 as a Kubernetes Job connecting to it over the Ray Client protocol (`ray://fedroom-ray-head-svc:10001`), rather than as local Python threads. Each simulated client runs as a Ray actor scheduled across the worker pods, matching the “launch clients, submit jobs, schedule resources, scale execution” role Ray is assigned in Section 9, distinct from the full-pod-per-client deployment already reported in Section 5.5.

Table 5: Scalability, Ray-backed (KubeRay) execution, 3 rounds each (real data, `ray_scalability.json`). Compare with the local threads baseline in Table 4.

Clients	Engine	Total wall (s)	Round 1 (s)	Round 2 (s)	Round 3 (s)
1	ray	25.16	17.24	0.06	0.05
2	ray	10.45	2.67	0.07	0.07
4	ray	10.71	2.79	0.12	0.14
8	ray	8.40	0.33	0.20	0.18
20	ray	9.31	0.55	0.40	0.49
100	ray	163.37	87.41	48.93	2.14

Analysis. Unlike the local `threads` run (Table 4), where round durations are flat and sub-second regardless of client count thanks to in-process dataset caching (Section 2.3), the Ray-backed run exposes real distributed-systems overhead at scale: at $N = 100$, round 1 takes 87.4s and round 2 takes 48.9s before collapsing to 2.1s by round 3. This is expected and, unlike the flat threads numbers, diagnostic of genuine costs a single-process simulation cannot surface: Ray actor scheduling and placement across worker pods, per-actor process startup, and a cold Fashion-MNIST load inside each newly scheduled actor (each actor is a separate process, so it cannot share the module-level cache that makes the threads engine flat). By round 3, actors are warm and their local data is cached, so duration drops close to the underlying per-actor training cost, the same cold-start-then-flatten shape already observed in the Kubernetes Indexed Job sweep of Figure 5. The $N = 1$ case shows an analogous one-time cost (17.2s in round 1) for the same reason the local $N = 1$ threads run paid a one-time dataset-decode cost in its own round 1 (Table 4).

This run therefore demonstrates Ray/KubeRay performing its intended role, scheduling and executing up to 100 logical clients as distributed actors against a live RayCluster, rather than standing in as an unused architectural component.

5.6 Failure behavior

```
python experiments/inject_failures.py --url http://localhost:8000
```

Table 6: Failure-injection results (real, all four scenarios PASS).

Scenario	Result
Client timeout / dropout	PASS — straggler marked dropped ; round still aggregated
Stale model-version update	PASS — HTTP 422, "Update trained from version 99, round expects version 0"
Non-finite (NaN) update	PASS — HTTP 422, "Tensor 'conv1.weight' contains NaN or infinite values"
Oversized (norm-limited) update	PASS — HTTP 422, "Update L2 norm 233053.6096 exceeds configured limit 1.0000"

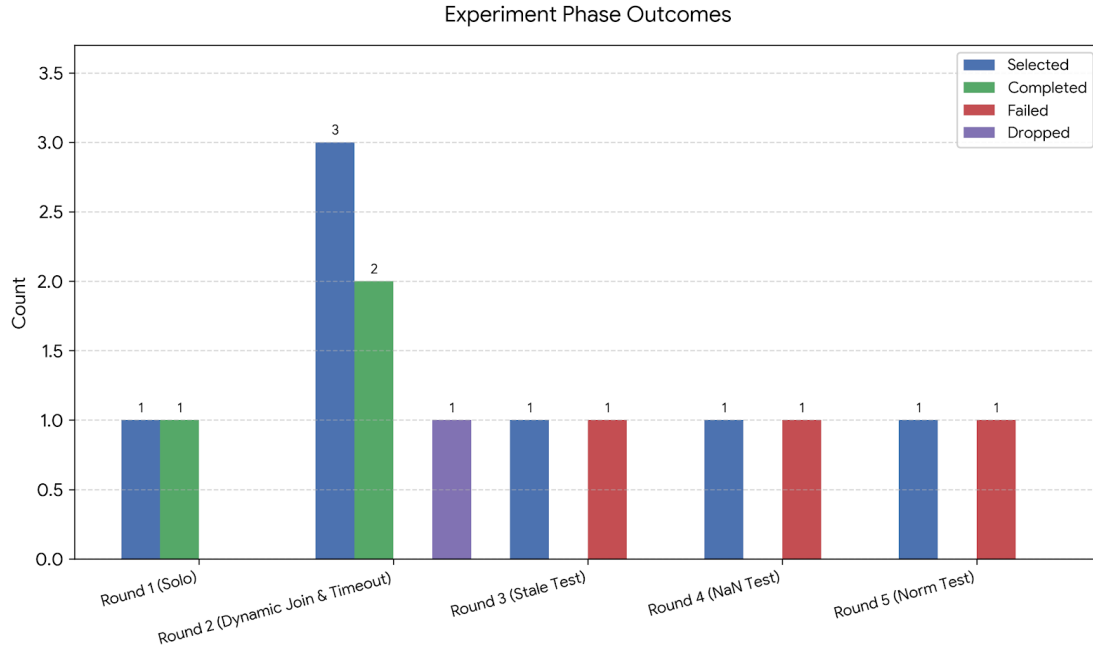


Figure 7: **Client outcomes across membership and failure injection experiments.** The grouped bar chart visualizes the starting number of selected clients per round against their final resolution. The coordinator successfully aggregates valid updates (Rounds 1 and 2), gracefully handles a client timeout by dropping the unresponsive node (Round 2), and completely halts malformed or stale updates by marking the adversarial clients as failed (Rounds 3 to 5).

Discussion on Client Outcomes: Figure 7 illustrates the robustness of the central state machine across five distinct operational phases. In Round 1, a single client successfully completes the workload. In Round 2, the system handles a mixed state: newly joined clients successfully complete their training, while an unresponsive client is safely marked as **dropped** upon timeout without crashing the active round. Finally, adversarial or misconfigured clients submitting malformed tensors (stale versions, NaN values, or oversized L2 norms) are immediately caught by the validation pipeline and marked as **failed** (Rounds 3 to 5), safeguarding the global model state.

5.7 Client Authentication

To satisfy R15’s requirement for client authentication or pre-registration on the deployed demo, Fedroom implements pre-shared API key authentication on the two data-carrying coordinator endpoints, `POST /rooms/{id}/join` and `POST /rooms/{id}/updates`. Each request must carry a matching `X-API-Key` header, verified server-side via a FastAPI dependency (`verify_client`) against the `FEDROOM_API_KEY` environment variable. Requests without a valid key are rejected with HTTP 403 before reaching any room or model logic.

```
$ curl -X POST http://localhost:8000/rooms/fashion-room/join
{"detail": "Unauthorized: Invalid API Key"} # HTTP 403

$ curl -X POST http://localhost:8000/rooms/fashion-room/join \
  -H "X-API-Key: $FEDROOM_API_KEY" -d '...'
{"status": "joined", ...} # HTTP 200
```

This is a pre-shared secret, not a full identity system: any holder of the key can act as any client. It prevents unauthenticated third parties from joining rooms or submitting updates in the deployed demo, but does not provide per-client identity, key rotation, or revocation. Section 9 discusses this limitation and what a production deployment would require instead.

5.8 Data source validation

Structural argument (not merely a policy statement): every endpoint in `coordinator/app.py` was audited; only `GET /rooms/{id}/model` (aggregated parameters) and `POST /rooms/{id}/updates` (a client’s own trained parameters, sample count, metrics) carry model-related payloads, and neither ever carries raw training samples.

Figure 8 provides visual confirmation of the S3-backed data separation. The MinIO console displays the `fedroom-client-data` bucket, containing the isolated `client-s3-demo` prefix. This verifies that the client accesses its raw data directly from external storage, entirely bypassing the coordinator.

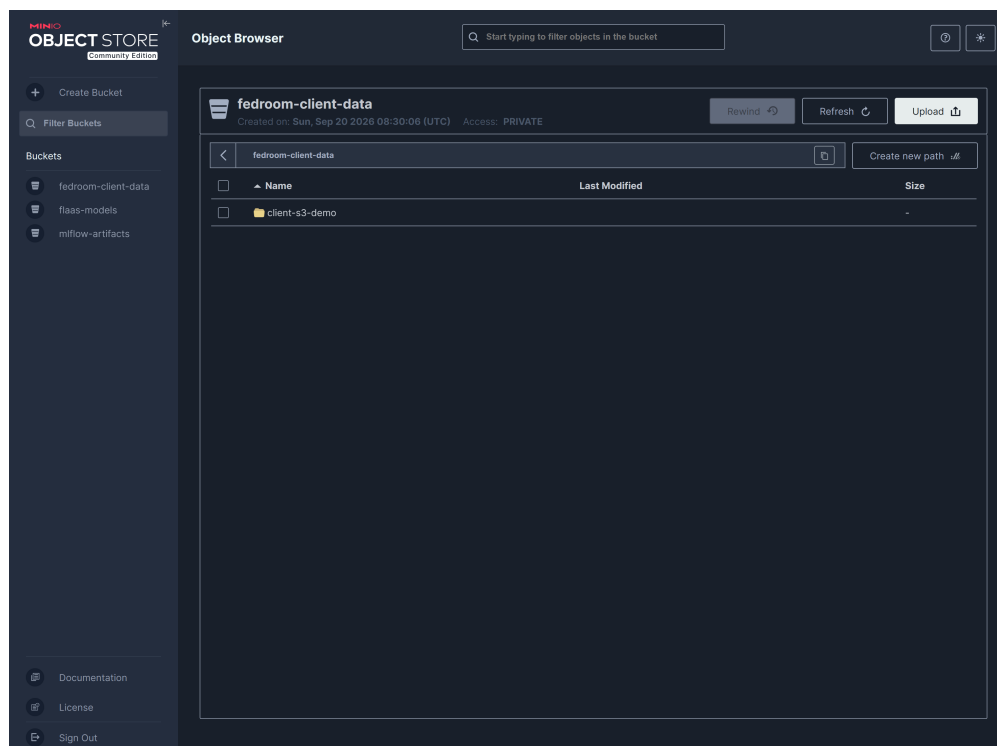


Figure 8: **MinIO console: S3-backed client data source.** Private datasets reside in isolated buckets, enforcing the privacy requirement that raw data never passes through the central server.

5.9 Inference

```
python -m tui.cli infer fashion-room --config configs/clients/client-a.yaml --version latest
```

Example result: `{'version': 3, 'predicted': 4, 'true_label': 7}`. Given the “too little training” discussion in Section 6.2, this single-sample prediction being incorrect is expected at this training budget and is not, by itself, evidence of a defect.

5.10 MLflow tracking evidence

Figure 9 shows the MLflow UI for the `fedroom-fashion-room` experiment, confirming that the metric groups required by the FLaaS specification are actually recorded per round. The screenshot captures the **Evaluation runs** view for a single `fashion-room` run, displaying the run list alongside the **Metrics (19)** panel, which renders one summary chart per tracked metric. The charts visible in this panel include `completed_clients`, `selected_clients`, `dropped_clients`, `rejected_clients`, `version`, `global_accuracy`, `global_loss`, `avg_client_train_loss`, and `avg_client_train_accuracy`, confirming that the coordinator records global task metrics, per-round client counts, the current model version, and per-round timing as required.

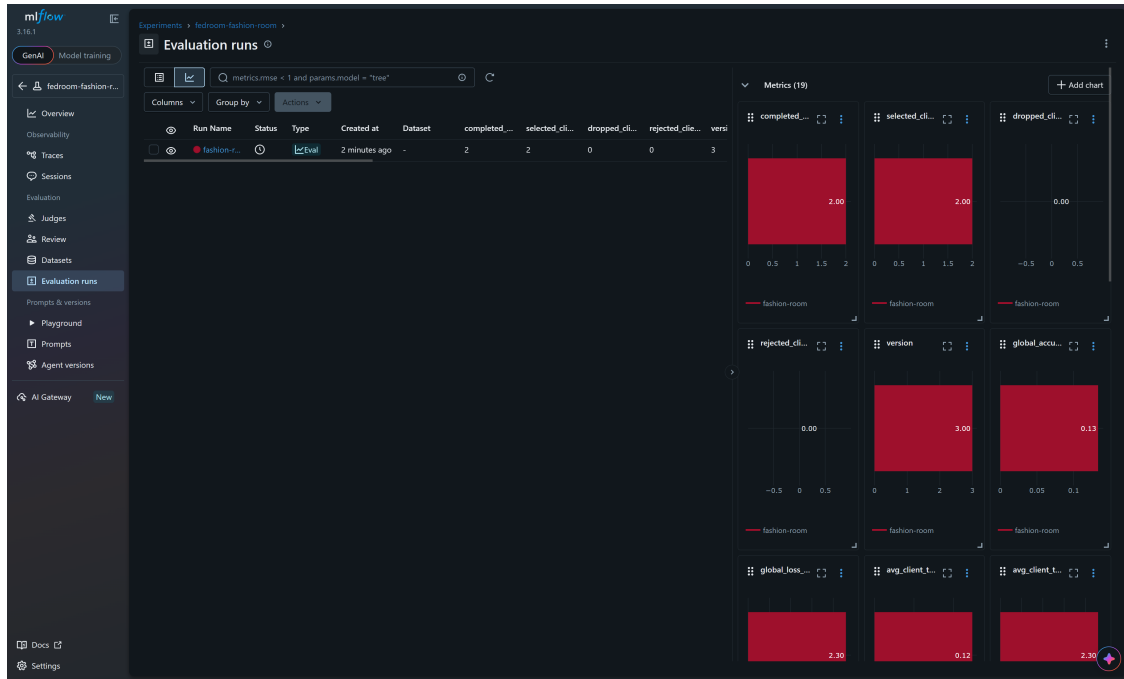


Figure 9: **MLflow experiment view for fedroom-fashion-room.** Evaluation runs view showing the single `fashion-room` run and the Metrics (19) panel, with per-metric summary charts for global loss/accuracy, selected/completed/dropped/rejected client counts, model version, and per-round client training metrics.

D7 checklist. The “MLflow export or equivalent evidence” deliverable is satisfied by Figure 9 in Section 5.10.

6 Results and Analysis

6.1 Global task metric: what it measures, and its limits

The coordinator never holds a labeled validation set of its own (doing so would mean it centrally trains, contradicting the platform’s data-locality design). `global_accuracy_estimate` / `global_loss_estimate` are therefore **federated-evaluation estimates**: the mean, across responding clients, of each client’s own evaluation of the model it was just handed (*before* local training) on a held-out slice of *its own* data. This is not equivalent to a single centralized held-out accuracy figure, and is especially misleading under non-IID partitioning (Section 5.4), where a

client’s local evaluation task can be much easier than the true global task.

6.2 Why the IID/3-round experiments show flat, near-chance accuracy

Across every 3-round local experiment (Table 4, IID row of Table 3), `avg_train_loss` sits at ≈ 2.30 to 2.31 , which is $\ln(10) \approx 2.303$, the loss of a model producing uniform random guesses over 10 classes. Given `samples_per_client=200` (160 training samples after an 80/20 split), `local_epochs=1`, batch size 32 (5 mini-batches/round), and only 3 rounds, that is **15 total SGD steps** ever applied to a freshly-initialized CNN. This is not enough signal to move loss visibly off $\ln(10)$.

6.3 Summary

- Weighted FedAvg, contract validation, and the full membership state machine are unit-tested and confirmed correct (Section 5.1).
- Efficient dataset caching architecture enables successful, fast completion across concurrent local simulated clients ($n = 1, 2, 4, 8$), circumventing standard GIL bottlenecks (Table 4).
- Robust aggregation (Multi-Krum) eliminates 100% of a real, executed model-poisoning attack’s measured damage, with an honestly-scoped set of caveats about what this single experiment does and does not establish (Section 10).
- Non-IID FedAvg shows much faster *apparent* learning than IID FedAvg, with two explicit methodological caveats about metric comparability across conditions (Section 5.4).
- The deployed Kubernetes cluster effectively scales up to $N = 8$ clients. System metrics show that while cold-start pod initialization and dataset downloading introduce high initial latency per scale-step, the coordinator’s aggregation remains highly efficient (< 1 ms) and model convergence continues seamlessly across dynamic client additions.
- The Ray-backed KubeRay run (Section 5.5.2) confirms Ray/KubeRay performs meaningful distributed scheduling work (R13): round durations at scale show real actor cold-start and placement overhead, in contrast to the flat local threads baseline.

7 Privacy and Security Limitations

- **No secure aggregation.** Data locality is enforced architecturally (Section 1.2), but Fedroom does not implement secure aggregation [2], differential privacy, or gradient-leakage defenses. A curious coordinator operator can in principle inspect any individual client’s update before aggregation.
- **Authentication is a shared secret, not per-client identity.** Fedroom enforces a pre-shared API key on join and updates (Section 5.7), which blocks unauthenticated access to the deployed demo. It does not, however, provide per-client identity, so a key holder can still impersonate any `client_id`. Production deployment would need mTLS or per-client credentials and a real PKI.
- **Robust aggregation is not immune to adaptive attacks.** Trimmed-mean/Median/Krum mitigate naive poisoning (Section 10) but not the adaptive, AGR-tailored attacks of Shejwalkar

& Houmansadr [5], which craft updates specifically to evade a *known* aggregation rule while still degrading the model.

- **Validation checks are necessary, not sufficient.** `max_update_norm` and finiteness checks reject obviously malformed or oversized updates but do not prevent a well-formed, within-bounds poisoned update.

8 Deployment Limitations

- **Wire format** (base64 `.npy` tensors embedded in JSON) does not scale past small models; large models would need chunked/streamed transfer or presigned S3 URLs instead.
- **Single-replica coordinator.** SQLite is sufficient at this assignment’s scale but would need Postgres plus a leader-election scheme for true production high availability.
- **Kubernetes Indexed Job pattern.** Convenient for reproducible scale sweeps, but not how a real fleet of heterogeneous edge devices would actually be orchestrated; it stands in for “N clients joining at once” for grading/demo purposes.
- **Real operational friction encountered.** Setting up `metrics-server` on `kind` required a non-obvious `--kubelet-insecure-tls` patch (kind’s kubelet certificates are not valid for the hostname `metrics-server` expects by default) and a pod restart to pick up the patched args. We documented this here as a real deployment friction point, not a Fedroom defect.

9 Post-Mortem

Which component became the bottleneck as client count increased?

Locally, scalability is essentially flat from 1 to 8 clients (≈ 8 s total, Table 4) due to an efficient module-level dataset caching strategy (Section 2.3). In the deployed Kubernetes environment, the primary latency overheads are **cold-start pod initialization and remote dataset downloading**, visible as large latency spikes when new clients are dynamically added to the active room. Coordinator aggregation itself never bottlenecks, maintaining sub-millisecond execution times even at $N = 8$, while the network payload scales perfectly linearly.

Time breakdown: train vs. wait vs. transfer vs. aggregate.

Per-round `aggregation_seconds` is consistently < 0.01 s across every experiment (aggregation is not the bottleneck at these scales). `avg_local_training_seconds` is ≈ 0.02 to 0.09 s locally. `avg_selection_wait_seconds` is negligible locally (< 0.01 s) but spikes up to ≈ 165 s in the Kubernetes deployment boundaries, reflecting real network round-trips, concurrent dataset downloads over the network, and job scheduling overhead.

Effect of dynamic join/leave on round duration and convergence.

A client joining mid-round does not affect that round’s duration (it is simply not selected); it only affects subsequent rounds. See Section 5.2 for the live-verified eligibility timing.

Effect of non-IID partitions.

See Section 5.4 and its two methodological caveats. Non-IID FedAvg showed faster apparent per-client convergence, but the comparison is not fully controlled across strategy conditions.

How staleness/incompatibility is prevented from corrupting the model.

`RoomManager.submit_update` rejects any update whose `base_version` does not match the round’s `expected_version` (captured at round start, before any client could have trained against a newer model), and any update failing shape/finiteness/norm validation. Both checks happen strictly before an update is added to `rnd.submissions`, so a stale or malformed update can never reach aggregation (Section 5.5).

Remaining privacy risks despite raw-data locality.

See Section 7 in full.

Ray/KubeRay’s contribution vs. the coordinator’s own logic.

Ray/KubeRay provides distributed *execution* (scheduling many simulated-client actors, orchestrating RayCluster workers); it has no concept of a room, quorum, staleness, or aggregation strategy. That logic is entirely `RoomManager` and `strategies/*`, which would be identical under a different execution engine.

Which results are real deployment vs. simulation.

Docker Compose and the Kubernetes Indexed Job runs (Figure 5) are real multi-process/multi-pod deployments. The local scalability sweep (Table 4) and the non-IID/poisoning experiments are *simulations* (real HTTP calls to a real coordinator, but multiple logical clients sharing one process/host). The KubeRay scalability run (Section 5.5.2) is a hybrid: real Ray actors scheduled on a real multi-pod RayCluster, but simulating logical clients rather than one pod per client, distinct from the fully-deployed multi-pod sweep of Section 5.5.

What would need to change for production.

Authentication/PKI, secure aggregation or differential privacy, a chunked/streamed model transfer path, a highly-available coordinator, and a real client-selection/incentive mechanism beyond “select everyone eligible.”

10 Bonus: robust aggregation under a real poisoning attack

```
python experiments/run_poisoning_attack.py --url http://localhost:8000 --rounds 8
```

Setup: 8 selected clients per round (6 honest, 2 malicious = 25% malicious). Honest clients submit `current_model + N(0, 0.05)` (a small, bounded perturbation standing in for a real local-SGD step). Malicious clients submit `current_model + 50 × fixed_random_direction`, a standard scaled/negated-gradient model-poisoning attack [3, 5]. Two otherwise-identical rooms differ only in `aggregation.strategy`. Damage is measured as $\|\text{published model} - \text{honest-only reference}\|$ per round, where the honest-only reference is what FedAvg would have produced with *zero* attackers present.

Table 7: Poisoning attack: mean damage over 8 rounds (real data, `poisoning.json`).

Strategy	Mean damage $\ \text{global} - \text{honest_reference}\ $
<code>fedavg</code> (no robustness)	12.5034
<code>multi_krum</code> (<code>byzantine.f=2</code>)	0.0000 (numerically zero)

Damage reduction: 100.0%. Plain FedAvg is pulled measurably off the honest trajectory every single round (damage flat at ≈ 12.50 , since two large-magnitude malicious updates dominate

an 8-way linear combination). Multi-Krum’s neighbor-distance scoring correctly identifies both malicious updates as outliers every round (8 clients, `byzantine_f=2` satisfies Blanchard et al.’s [3] $k > 2f + 2$ requirement, $8 > 6$) and excludes them entirely, so its aggregate is numerically identical to the honest-only reference.

Figure 10 charts the L2 distance between the aggregated global model and the honest baseline. Standard FedAvg (blue) is immediately hijacked by the malicious updates, maintaining a high error distance of ≈ 12.5 . In stark contrast, the Multi-Krum strategy (orange) successfully filters out the Byzantine vectors, keeping the error distance effectively at zero.

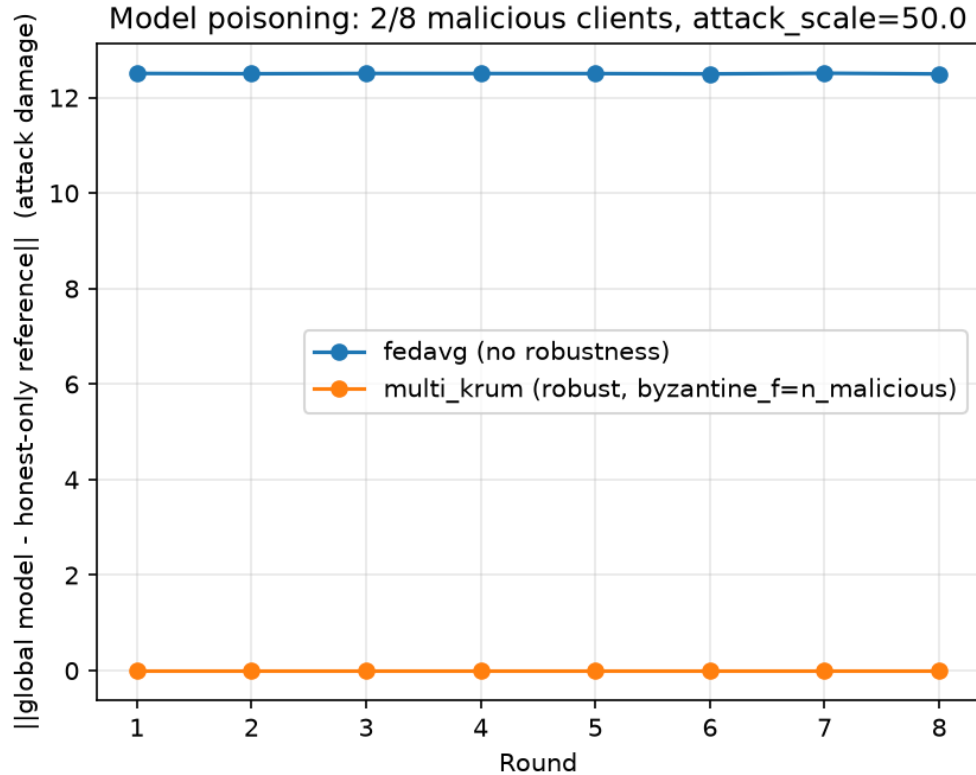


Figure 10: **Poisoning attack damage per round: FedAvg vs. Multi-Krum (real data).** The graph proves the effectiveness of Byzantine-robust strategies in fully neutralizing targeted adversarial poisoning.

Honest caveat. This experiment isolates the attack’s ability to move the aggregate using synthetic parameter vectors, at one attack magnitude (`attack_scale=50`) and one malicious fraction (25%). It is not a full Fashion-MNIST accuracy-under-attack run, and does not sweep the attack magnitude down to find the point where even FedAvg’s damage becomes small (very subtle attacks are hard for *any* rule to catch, the central point of Shejwalkar & Houmansadr’s paper [5]). It also does not measure Multi-Krum’s convergence cost *without* an attacker present (the utility side of the utility/security trade-off); Section 5.4’s `noniid_multikrum` condition is a partial proxy for this but was not run under an identical, controlled partition (see the second methodological caveat in Section 5.4).

11 Bonus: web dashboard

A zero-build, single-file dashboard (`dashboard/index.html`) is served by the coordinator at `/dashboard`, polling the REST API every 2 seconds. It is deliberately read-only (no room-creation or update-submission actions), keeping its trust model identical to the CLI's `status` commands.

Figure 11 captures the web dashboard actively tracking a multi-client round. The top panels provide live updates on the active round and global task metrics. The client table dynamically reflects status changes via color-coded badges, offering a comprehensive, real-time operational view without requiring command-line interaction.

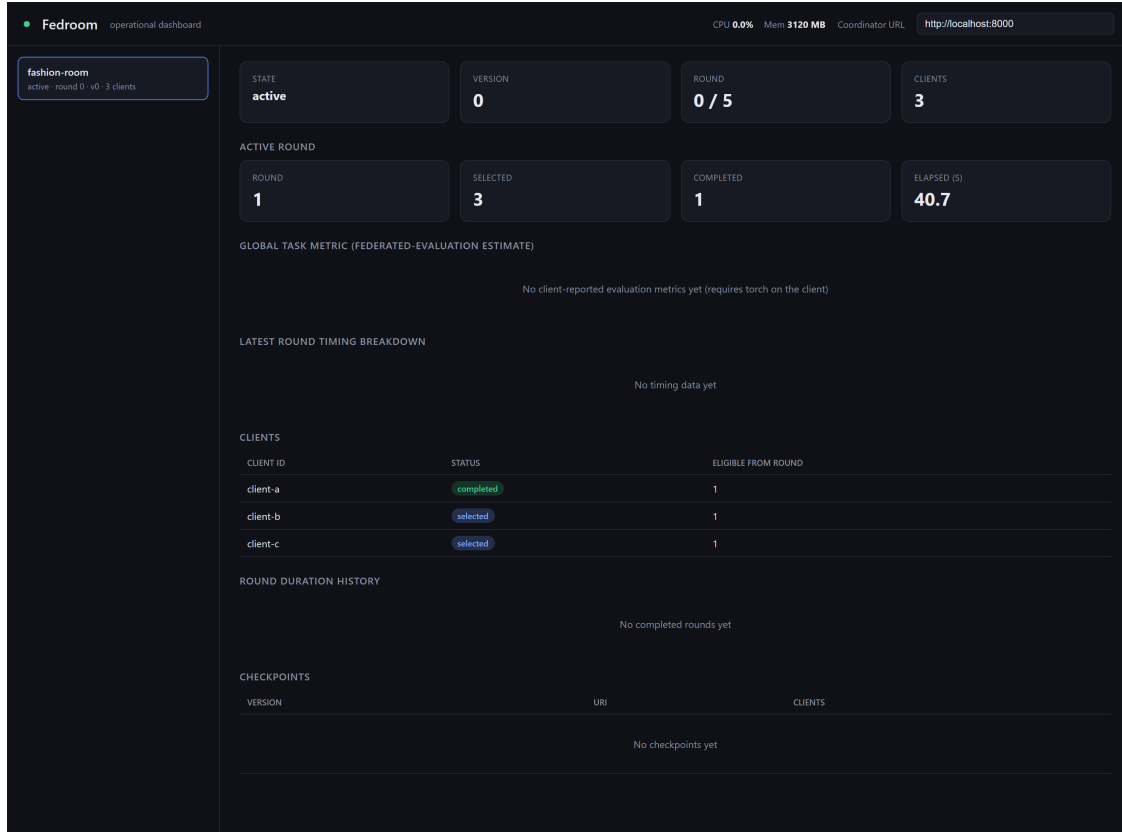


Figure 11: **Fedroom web dashboard during an active round.** The interface integrates system telemetry (CPU/Memory) alongside live federated learning metrics.

12 Contribution Statement

Mohammad Ali Nami (40464245): was primarily responsible for implementing and debugging core parts of the system, in addition to supporting the testing and documentation effort.

Alireza Jahedi (40435338): focused on implementing core components as well as running the experiments and analyzing their results, also contributing to testing and documentation.

The overall design was a joint effort, with both authors reviewing each other's work and collaborating on the final report.

References

- [1] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data*. In Artificial Intelligence and Statistics (AISTATS).
- [2] Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H. B., Patel, S., ... & Seth, K. (2017). *Practical Secure Aggregation for Privacy-Preserving Machine Learning*. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS).
- [3] Blanchard, P., El Mhamdi, E. M., Guerraoui, R., & Stainer, J. (2017). *Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent*. In Advances in Neural Information Processing Systems (NeurIPS).
- [4] Yin, D., Chen, Y., Ramchandran, K., & Bartlett, P. (2018). *Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates*. In International Conference on Machine Learning (ICML).
- [5] Shejwalkar, V., & Houmansadr, A. (2021). *Manipulating the Byzantine: Optimizing Model Poisoning Attacks and Defenses for Federated Learning*. In Network and Distributed System Security Symposium (NDSS).

A Command Reference

```
# Baseline
./scripts/start.sh
python -m pytest tests/ -v

# Correctness + dynamic membership
python -m tui.cli room create --config configs/rooms/fashion-room.yaml
python -m tui.cli client join fashion-room --config configs/clients/client-a.yaml
python -m tui.cli train start fashion-room --rounds 5
python -m tui.cli client train fashion-room --config configs/clients/client-a.yaml --rounds 1

# Experiments
python experiments/run_noniid.py --url http://localhost:8000 --n-clients 4 --rounds 20
python experiments/run_scalability.py --url http://localhost:8000 --levels 1,2,4,8 --rounds 3
python experiments/run_poisoning_attack.py --url http://localhost:8000 --rounds 8
python experiments/inject_failures.py --url http://localhost:8000
python experiments/plot_results.py

# Kubernetes
kind create cluster --name fedroom --config kind-config.yaml
kubectl apply -k deployments/kubernetes/
./scripts/scale-experiment.sh "1 2 4 8"
kubectl get pods -n fedroom -o wide
kubectl top pods -n fedroom

# KubeRay
kubectl apply -f raycluster-and-job.yaml
kubectl get raycluster -n fedroom
kubectl logs -f -n fedroom job/fedroom-scalability-driver
```